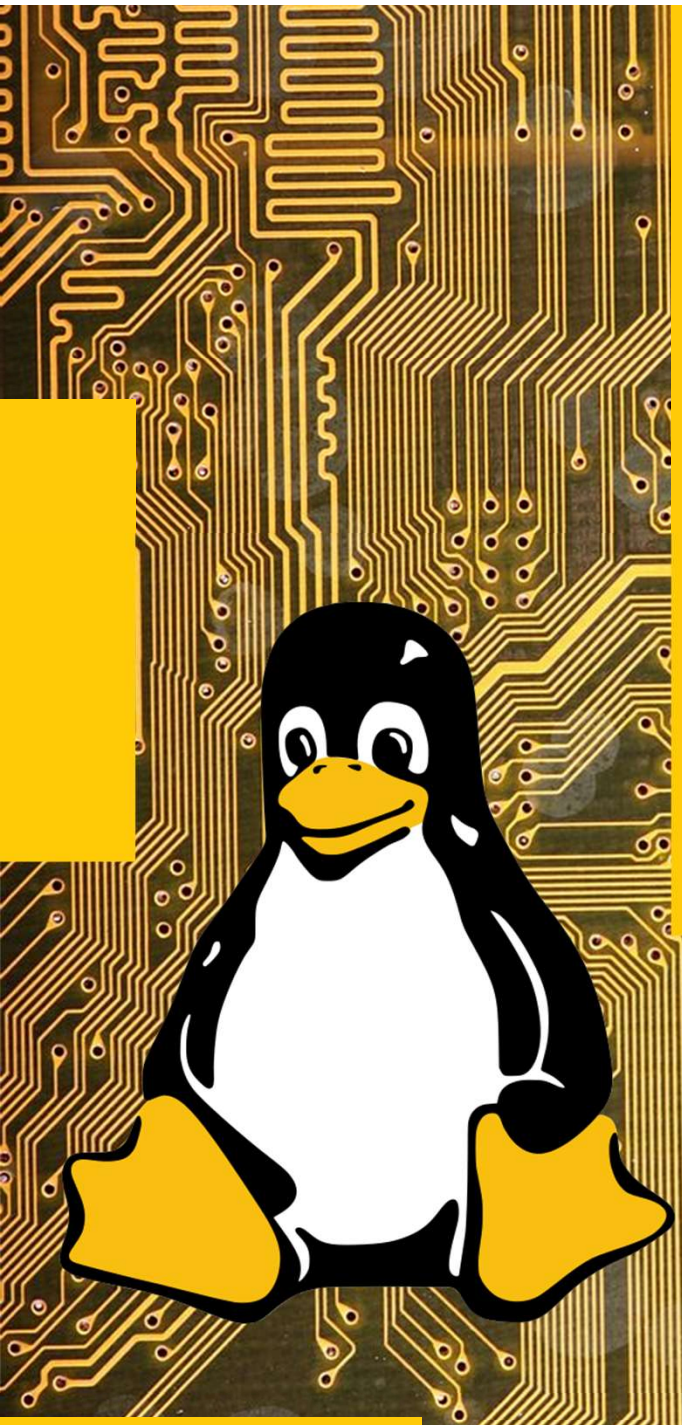


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

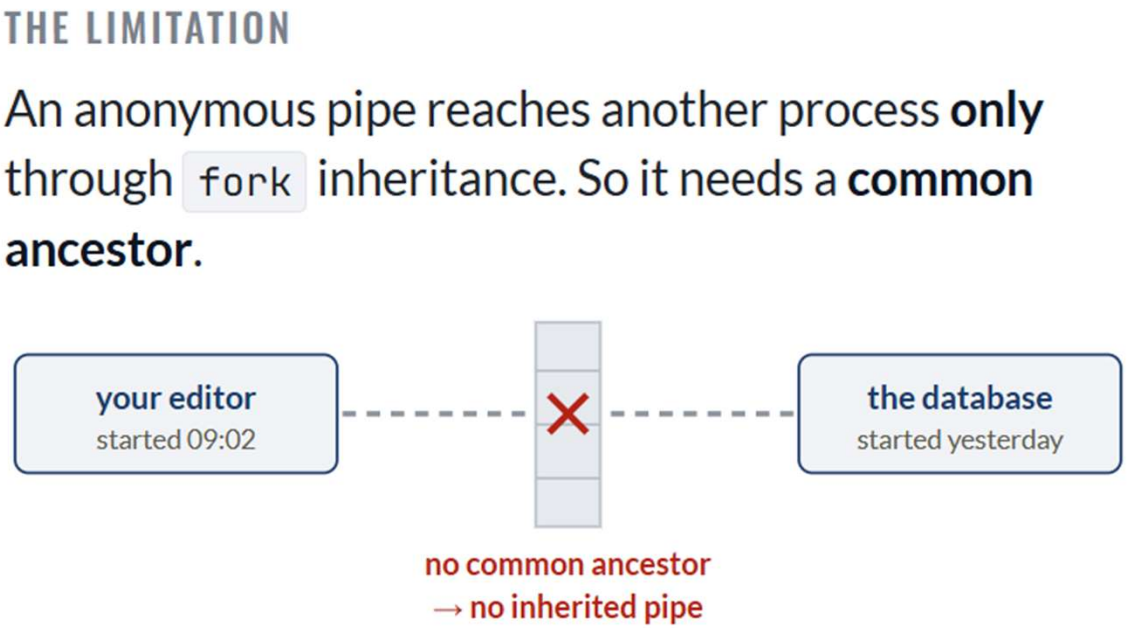
Inter-Process Communication FIFO and pipeline

INSTRUCTOR: Jalolliddin Yusupov

WEEK III Course Topics & Weekly Lecture Breakdown

Lecture Topic:
Inter-Process Communication - Pipes and FIFOs (TLPI Ch 44)

- Why processes cannot see each other's memory; message passing and shared memory. Pipes: creation, inheritance across fork, closing unused ends, blocking and end-of-file rules, atomic writes, the SIGPIPE signal; redirection with dup2. Named pipes (FIFOs) and a simple client-server built on them; the multi-stage pipeline.



Give the Pipe a Name (Named Pipe)

Same pipe semantics you already know — but now the pipe has a **name in the filesystem**, so unrelated processes can meet at it.

- **FIFO** = First In, First Out → the historical name for "named pipe"
- It is a **file** — but a special one: type `p` (pipe), not a regular file
- You open it with `open()`, just like any file
- Once open, you `read()` / `write()` it — **identical semantics to** `pipe()`
- Any process with the path and the right permissions can use it — **related or not**
- Works across terminals, users, scripts, and languages

> **ONE-LINE SUMMARY**
>
> **FIFO = pipe + a name.** Everything you learned about pipes still applies after the `open()`.

Give the Pipe a Name (Named Pipe)

The C API

```
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
// returns 0 on success, -1 on error (errno = EEXIST if it already exists)
```

The shell

Size is always 0. No data is ever stored on disk

```
$ mkfifo /tmp/myfifo
$ mkfifo -m 0600 /tmp/myfifo      # explicit permissions
$ ls -l /tmp/myfifo
  prw-r--r-- 1 alice alice 0 Nov 12 10:04 /tmp/myfifo
  ^
  └── 'p' means pipe (a directory 'd', a regular file '-')
```

The heart of it: `open()` semantics

This is the **only genuinely new idea** compared to pipe(). Everything else is the same.

Call	Blocks?	What happens
`open(path, O_RDONLY)`	**Yes**	Waits until some process opens the FIFO for writing
`open(path, O_WRONLY)`	**Yes**	Waits until some process opens the FIFO for reading
`open(path, O_RDONLY | O_NONBLOCK)`	No	Returns immediately, even with no writer present
`open(path, O_WRONLY | O_NONBLOCK)`	No	Returns `-1` with `errno = ENXIO` if no reader
`open(path, O_RDWR)`	No	Linux allows it, but it is **non-portable** and you will **never see EOF** — avoid it

REMEMBER

`open()` **is the handshake.** Both sides must arrive before data can flow.

CRUCIAL POINT

`mkfifo()` only **creates the name**. It does not open it, does not create a connection, and does not transfer any data. Opening is a separate step — and that step is where everything interesting happens.

Example: the writer/client

```c
/* writer.c — creates the FIFO and sends one message */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO_PATH "/tmp/demo_fifo"

int main(void){
    if (mkfifo(FIFO_PATH, 0666) == -1 && errno != EEXIST) {
        perror("mkfifo");
        exit(EXIT_FAILURE);
    }
    printf("writer: waiting for a reader (open blocks)...\n");
    int fd = open(FIFO_PATH, O_WRONLY);      /* < blocks here */
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }
    printf("writer: reader arrived, sending\n");
    char *msg = "hello over FIFO\n";
    if (write(fd, msg, strlen(msg)) == -1) perror("write");
    close(fd);
    return 0;
}
```

`errno ≠ EEXIST`

"A FIFO that is already there is fine; any *other* failure is not." Without this test a restart of the server dies on its own leftover well.

The **perror() function** is a built-in C standard library function used to print a human-readable error message to the standard error stream (**stderr**).

Example: the reader/server

```c
/* reader.c — receives everything until the writer closes */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#define FIFO_PATH "/tmp/demo_fifo"
int main(void)
{
    printf("reader: waiting for a writer (open blocks)...\n");
    int fd = open(FIFO_PATH, O_RDONLY);      /* < blocks here */
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }
    printf("reader: writer arrived\n");
    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        write(STDOUT_FILENO, buf, n);
     /*n==0>all writers closed (EOF)*/
    printf("reader: EOF, done\n");
    close(fd);
    //unlink(PATH); to remove the FIFO
    return 0;
}
```

Pure shell, no C at all

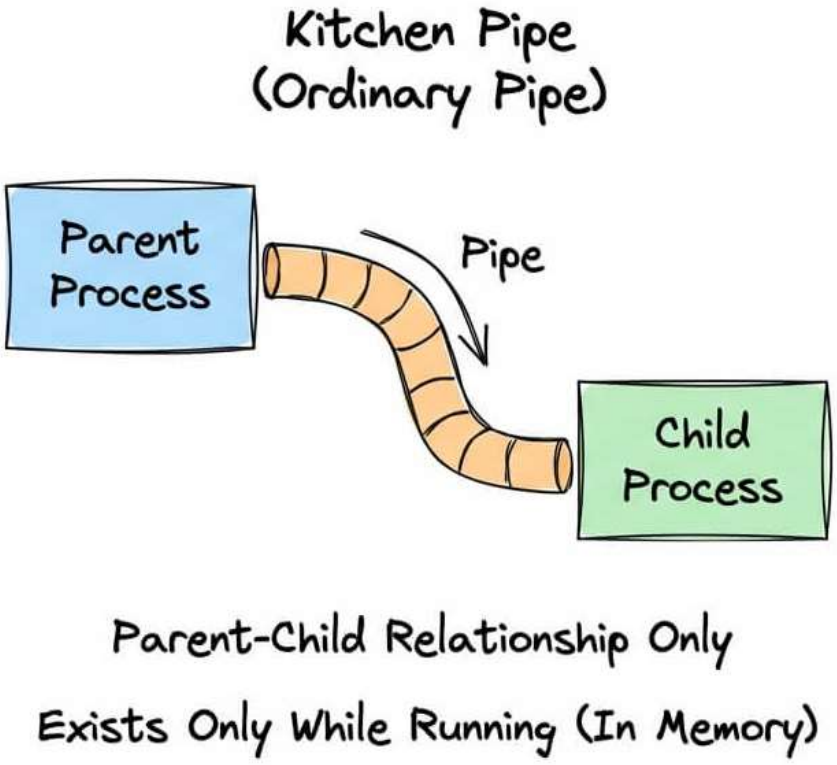
```
# terminal 1
$ cat /tmp/myfifo

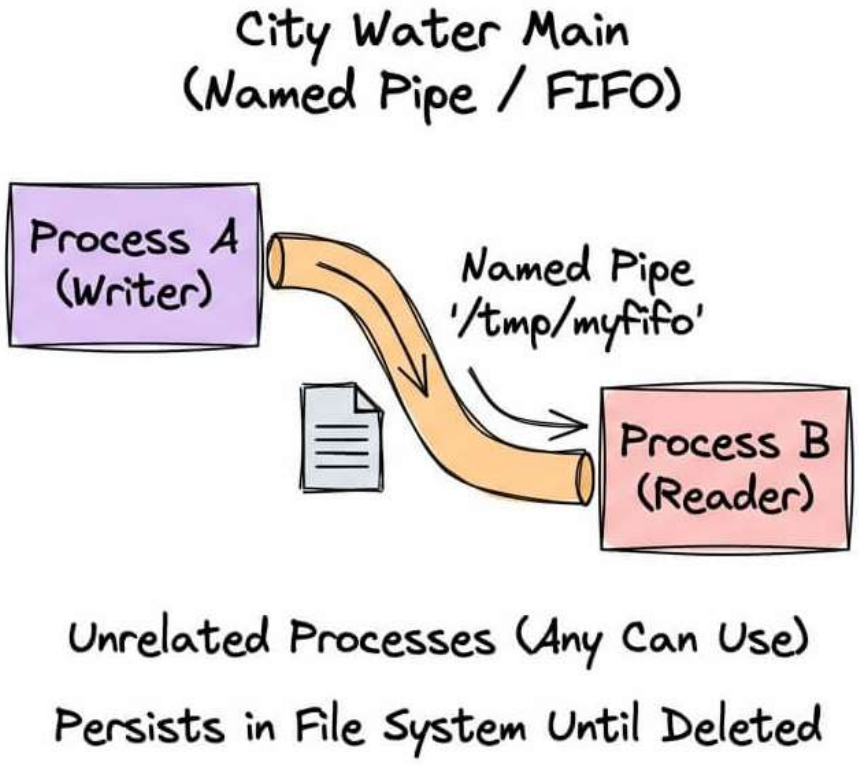
# terminal 2
$ echo "hello from bash" > /tmp/myfifo
```

Homework

Write two completely unrelated programs that exchange 5 messages over a FIFO, alternating turns. (Hint: you need two FIFOs, or a strict protocol.)

Summary



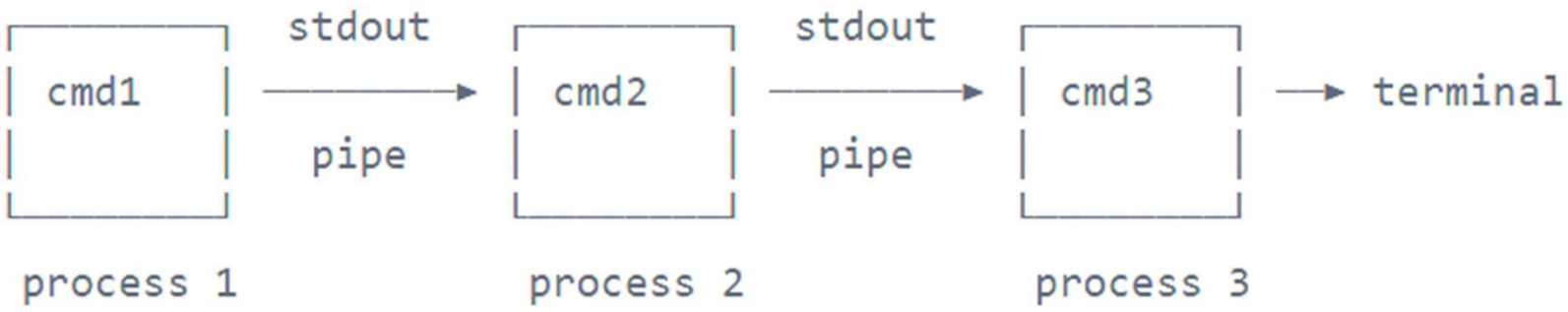


What is a pipeline?

You know `pipe()`. You know FIFOs. Now: how does the shell turn `cmd1 | cmd2 | cmd3` into a chain of cooperating processes?

```
ls -l | grep ".c" | sort | wc -l
```

- Run all commands **concurrently**, as separate processes.
- Connect stdout of each to stdin of the next.
- Data flows left → right, as a **stream of bytes**.
- No temporary files, no buffering of the whole output, no coordination code.
- Each program just reads stdin and writes stdout — it does not know it is in a pipeline.





Questions?